

Project Design Phase-II Technology Stack (Architecture & Stack)

Date	18 June 2026
Team ID	
Project Name	ShopEZ-E-commerce-Application
Maximum Marks	4 Marks

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

Example:

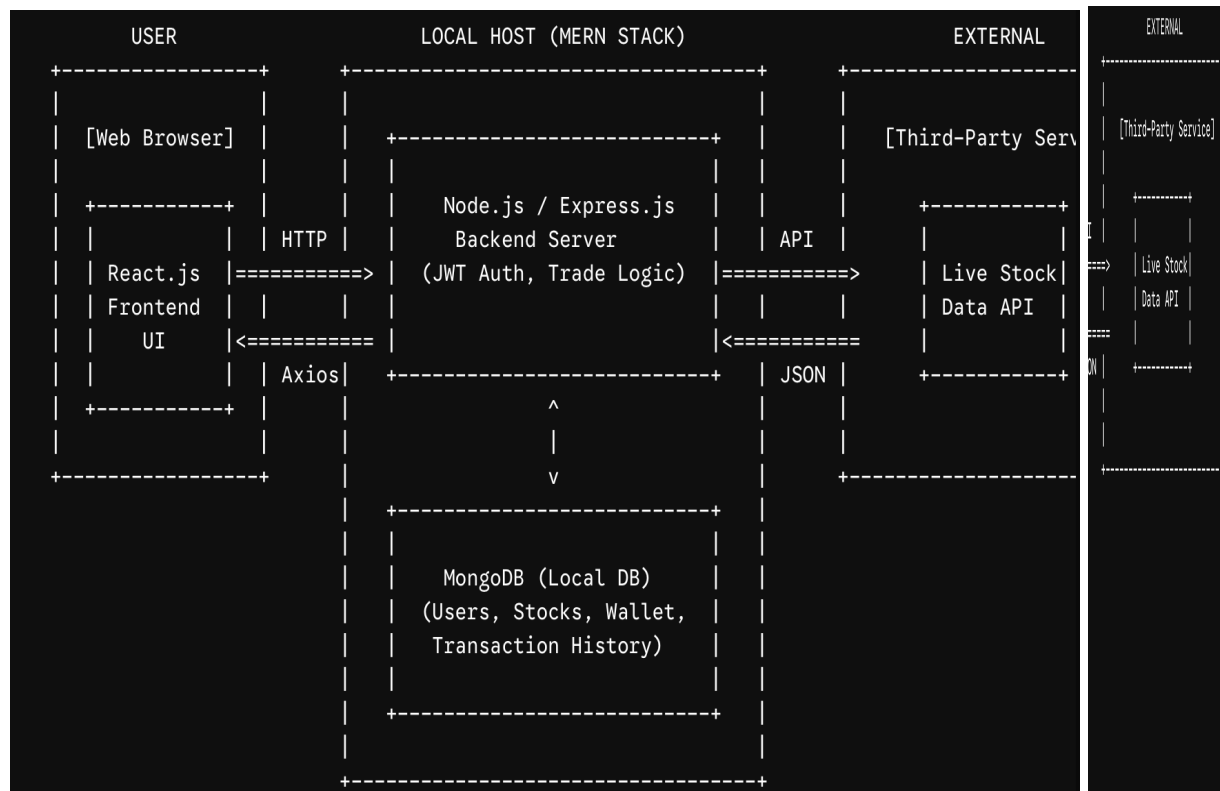


Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	Web-based UI for users to browse stocks, view portfolios, and execute virtual trades..	React.js, HTML5, CSS3, Bootstrap / Material-UI, Chart.js (or Recharts)
2.	Application Logic-1 (Routing & APIs)	Backend framework managing RESTful API routes, user requests, and data flow.	Node.js with Express.js

3.	Application Logic-2 (Auth & Security)	Logic for user registration, login, and secure session management.	JWT (JSON Web Tokens), bcrypt.js
4.	Application Logic-3 (Market Data)	Logic to fetch, process, and serve live stock data to the frontend.	Axios (HTTP client for API requests)
5.	Database	Local database storing Users, Stocks, Transactions, and Portfolio collections.	MongoDB (Local Instance), Mongoose (ODM)
6.	Cloud Database	N/A	N/A
7.	File Storage	Local storage for project assets, env files, and component logs.	Local Filesystem
8.	External API-1	Real-time stock market data provider to populate the trading dashboard.	Alpha Vantage API / Polygon.io API (or similar financial API).
9.	External API-2	Not currently required for the core MVP.	N/A
10.	Machine Learning Model	Not applicable for this project scope.	N/A.
11.	Infrastructure (Server / Cloud)	Application development and testing environment.	Localhost (Node.js runtime environment).

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	Core frameworks driving the frontend and backend architecture.	React.js (Frontend library), Express.js (Backend framework)
2.	Security Implementations	Access controls to protect user data and ensure secure virtual trading.	JWT for route protection, bcrypt.js for password hashing, CORS middleware
3.	Scalable Architecture	The system uses a modular MVC (Model-View-Controller) structure, separating database models, routing, and UI.	MERN Stack (Modular Monolith)
4.	Availability	The application relies on the stability of the local Node server and the uptime of the 3rd party Stock API.	Local Node Server (nodemon), Axios for API handling
5.	Performance	Efficient state management in the UI and optimized database querying to handle rapid transaction logging.	React State/Hooks, MongoDB Indexing on frequent queries (e.g., Stock Symbols).